

Arquitetura de Computadores



Universidade
Christus

Unidade 05: Aritmética de Computador

Daniel Teixeira

prof.danielnt@gmail.com
dant25.github.io/professor

Motivação

- ULA
 - Executa as operações aritméticas e lógicas sobre os dados.
 - De certo modo, a ULA constitui o núcleo ou a essência de um computador.





Motivação

- UC, registradores, memória e E/S
 - Servem principalmente para trazer os dados a serem processados pela ULA e receber os resultados das operações efetuadas.

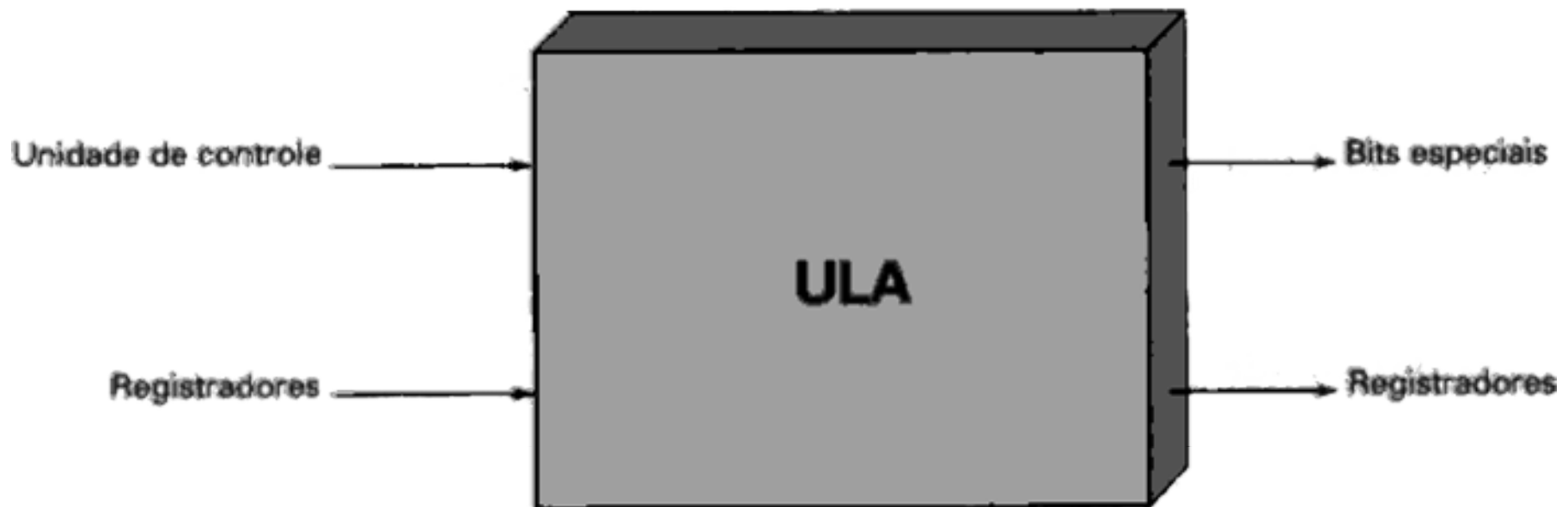
Motivação

- Composição da ULA
 - Dispositivos lógicos digitais simples,
 - Capazes de armazenar dígitos binários
 - Efetua operações simples de lógica booleana



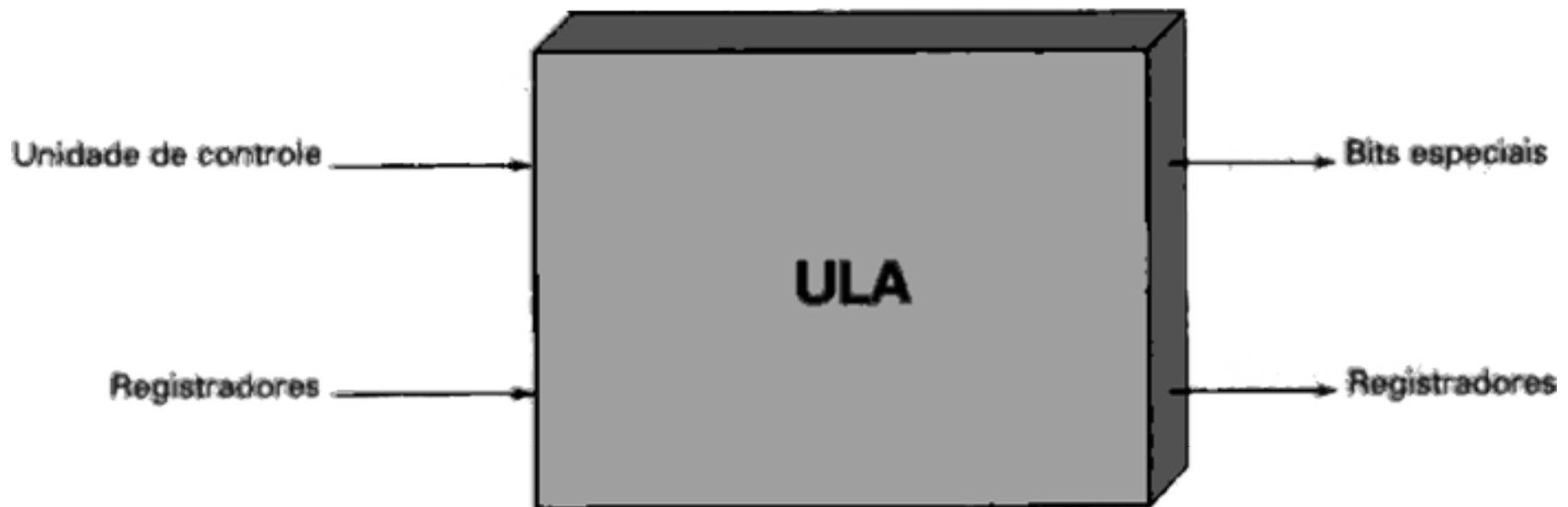
Motivação

- UC
 - Define a operação que a ULA irá realizar



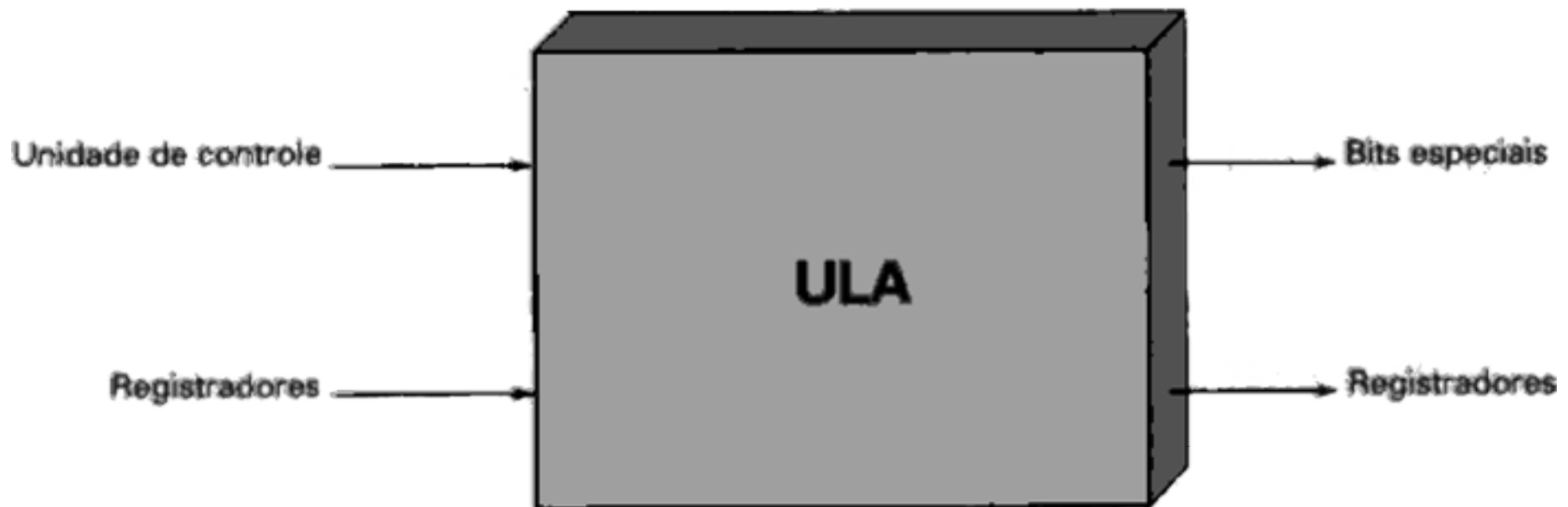
Motivação

- Registradores
 - Dados de entrada são fornecidos em registradores
 - Resultados das operações são armazenadas em registradores



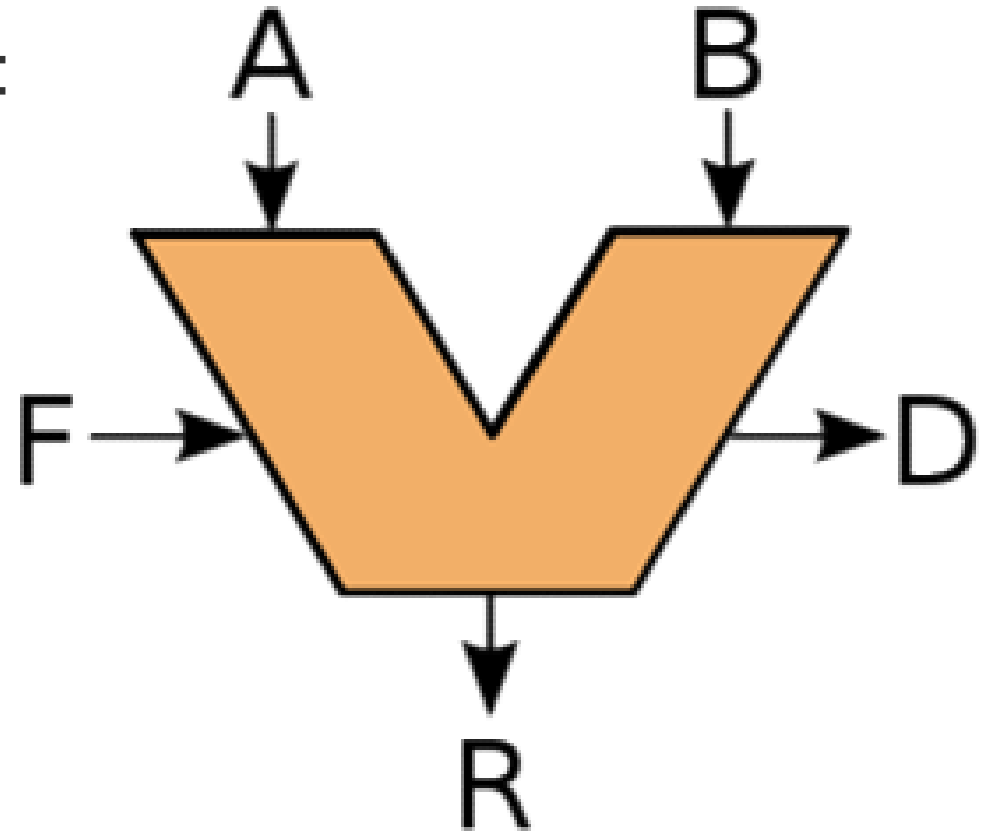
Motivação

- Bits especiais (flags)
 - Indicam um resultado de alguma operação
 - Ex: Overflow numa soma
 - Registrador overflow = 1



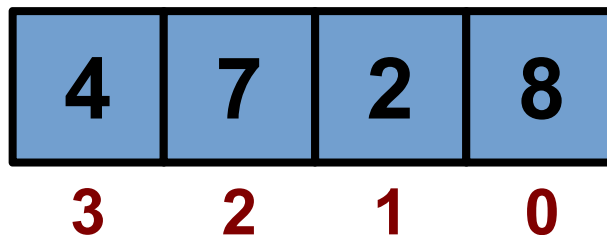
Motivação

- Representação da ULA:
 - **Entrada:** A e B
 - **Controle:** F
 - **Saída:** R
 - **Status:** D



Motivação

- Sistema de Numeração Decimal
 - **$X = \text{mantissa} * \text{base}^{\text{expoente}}$**
 - $83_{10} = (8 * 10) + (3 * 1)$
 - $4728_{10} = (4 * 1000) + (7 * 100) + (2 * 10) + (8 * 1)$
 - $4728_{10} = (4 * 10^3) + (7 * 10^2) + (2 * 10^1) + (8 * 10^0)$

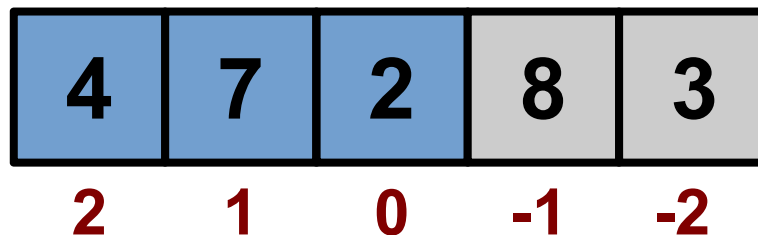


Motivação

- Sistema de Numeração Decimal **Fracionário**

- $472,83_{10} = (4 * 10^2) + (7 * 10^1) + (2 * 10^0) + (8 * 10^{-1}) + (3 * 10^{-2})$

$$X = \sum_{i=0}^{n-1} x_i * 10^i$$



Motivação

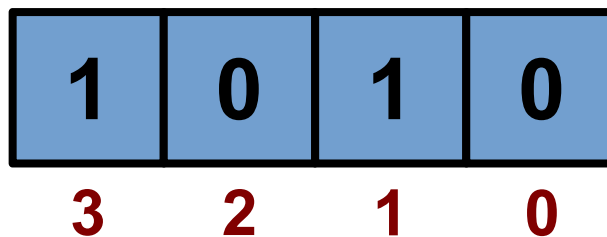
- Sistema de Numeração Binário

- $10_2 = (1 \cdot 2^1) + (0 \cdot 2^0) \rightarrow 2_{10}$

- $11_2 = (1 \cdot 2^1) + (1 \cdot 2^0) \rightarrow 3_{10}$

- $100_2 = (1 \cdot 2^2) + (0 \cdot 2^1) + (0 \cdot 2^0) \rightarrow 4_{10}$

$$X = \sum_{i=0}^{n-1} x_i \cdot 2^i$$



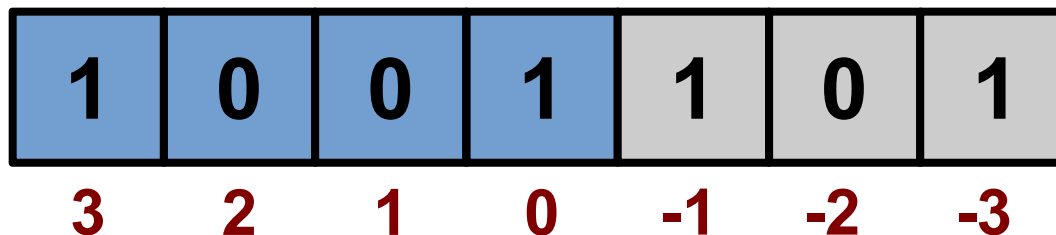
Motivação

- Sistema de Numeração Binário **Fracionário**

- $1001,101_2 = 2^3 + 2^0 + 2^{-1} + 2^{-3} \rightarrow 9,625_{10}$

- $2^{-1} = \frac{1}{2} = 0,5$

- $2^{-3} = \quad \quad \quad = 0,125$



Representação de Números Inteiros

- Representamos números negativos com o sinal –
- Para números fracionário usamos ,
 - $-1101,0101_2 = -13,3125_{10}$
- Como processar estes números no computador?
- É possível usar os sinais – e , ?

Representação de Números Inteiros

- **Representação sinal-magnitude**
 - Usar o bit mais significativo (mais à esquerda) para indicar o sinal (sinal-magnitude)
 - 0 → positivo
 - 1 → negativo
 - Exemplos:
 - +18 = **00010010**
 - - 18 = **10010010**

Representação de Números Inteiros

- Representação sinal-magnitude
 - N bits
 - N-1 bits para representar a magnitude
 - 1 bit para sinal-magnitude

Sinal-magnitude:

$$A = \begin{cases} \sum_{i=0}^{n-2} 2^i a_i & \text{se } a_{n-1} = 0 \\ -\sum_{i=0}^{n-2} 2^i a_i & \text{se } a_{n-1} = 1 \end{cases}$$

Representação de Números Inteiros

- Representação sinal-magnitude
 - Desvantagens:
 - 1) Menos 1 bit de dados
 - 2) Duas representações para o Zero:
 - $+010 = 00000000$
 - $-010 = 10000000$

Representação de Números Inteiros

- Representação sinal-magnitude
 - Desvantagens:
 - 3) Operações matemáticas precisarão considerar o **sinal-magnitude** e a **magnitude** nas operações

Representação de Números Inteiros

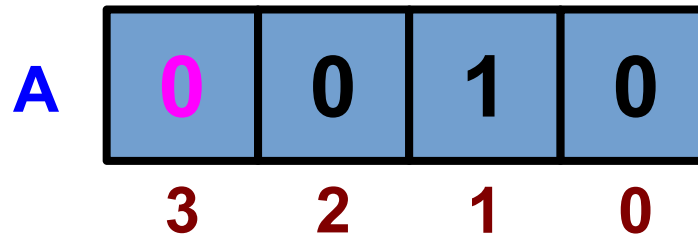
- Representação sinal-magnitude
 - Esta representação é raramente usada na implementação da parte inteira de uma ULA
 - Muitas desvantagens 

Representação de Números Inteiros

- **Representação em complemento de dois**
 - Usa o bit mais significativo como bit de sinal
 - Fácil testar se um número é positivo ou negativo
 - Demais bits são interpretados de maneira diferente

Representação de Números Inteiros

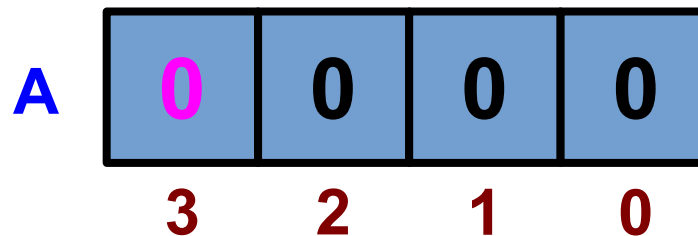
- Representação em complemento de dois



- **A** é um número inteiro com 4 bits na representação em complemento de dois
 - Bit (n-1) indica o sinal:
 - 0 → positivo
 - 1 → negativo

Representação de Números Inteiros

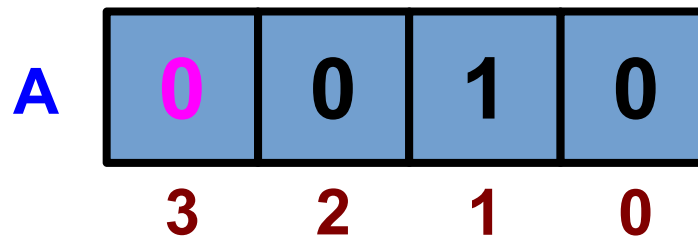
- Representação em complemento de dois



- Valor zero é tratado como um número positivo
 - Bit de sinal = 0
 - Todos os demais bits (magnitude) = 0

Representação de Números Inteiros

- Representação em complemento de dois

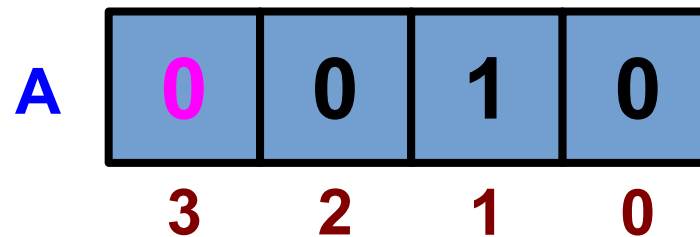


- Quando o bit (n-1) indicar um valor positivo
 - **Bits restantes** representam a magnitude do número

$$A = \sum_{i=0}^{n-2} a_i * 2^i$$

Representação de Números Inteiros

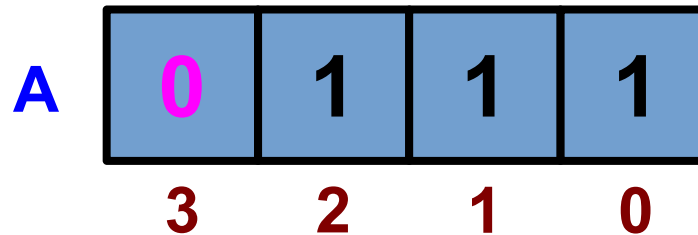
- Representação em complemento de dois



- $A = (0 * 2^3) + (0 * 2^2) + (1 * 2^1) + (0 * 2^0) = +2_{10}$

Representação de Números Inteiros

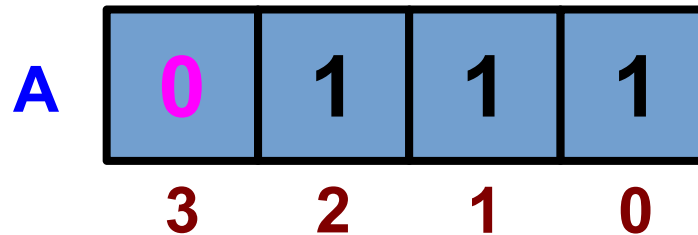
- Representação em complemento de dois



- Para 4 bits, quantos e quais valores **positivos** são possíveis representar usando complemento de dois?

Representação de Números Inteiros

- Representação em complemento de dois



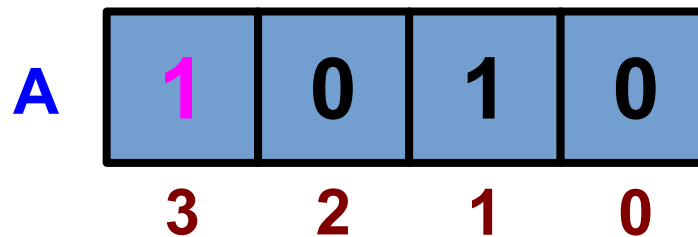
- Para 4 bits, quantos e quais valores **positivos** são possíveis representar usando complemento de dois?

- 2^{n-1} valores = 2^3 valores = 8 valores

- $0_{10} \rightarrow (2^{n-1} - 1)_{10}$ $0_{10} \rightarrow (2^3 - 1)_{10}$
 $0_{10} \rightarrow 7_{10}$

Representação de Números Inteiros

- Representação em complemento de dois



- Formulação geral é:

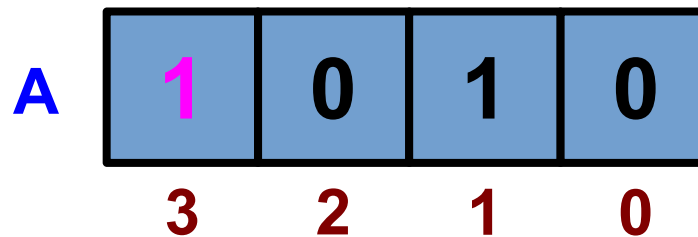
$$A = -2^{(n-1)} * a_{(n-1)} + \sum_{i=0}^{n-2} a_i * 2^i$$

- Positiva: $A = \sum_{i=0}^{n-2} a_i * 2^i$

- Negativa: $A = -2^{(n-1)} + \sum_{i=0}^{n-2} a_i * 2^i$

Representação de Números Inteiros

- Representação em complemento de dois

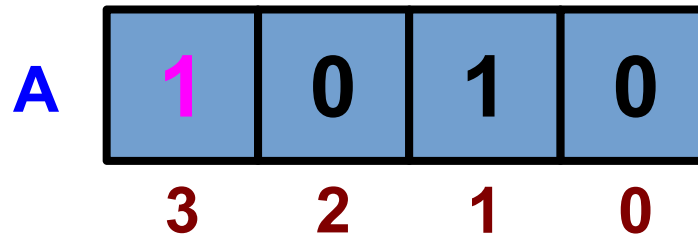


- Qual o valor de A?

$$A = -2^{(n-1)} + \sum_{i=0}^{n-2} a_i * 2^i$$

Representação de Números Inteiros

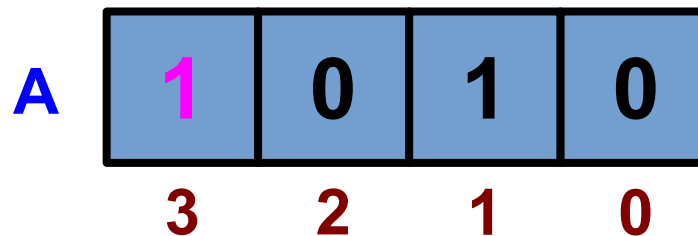
- Representação em complemento de dois



- Qual o valor de A?
 - $-8 + (0 * 2^2) + (1 * 2^1) + (0 * 2^0)$
 - $-8 + 2 = -6_{10}$

Representação de Números Inteiros

- Representação em complemento de dois



– Qual o valor de A?

- Tira o complemento de A

$$\begin{array}{ccccccc} \text{–} & 0 & 1 & 0 & 1 & = & (1 * 2^2) + (1 * 2^0) = \\ & 5_{10} & & & & & \end{array}$$

- Negação do complemento de A + 1

$$\text{– } - (5+1) = -6_{10}$$

1998, 1999, 2000, 2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025, 2026, 2027, 2028, 2029, 2030, 2031, 2032, 2033, 2034, 2035, 2036, 2037, 2038, 2039, 2040, 2041, 2042, 2043, 2044, 2045, 2046, 2047, 2048, 2049, 2050, 2051, 2052, 2053, 2054, 2055, 2056, 2057, 2058, 2059, 2060, 2061, 2062, 2063, 2064, 2065, 2066, 2067, 2068, 2069, 2070, 2071, 2072, 2073, 2074, 2075, 2076, 2077, 2078, 2079, 2080, 2081, 2082, 2083, 2084, 2085, 2086, 2087, 2088, 2089, 2090, 2091, 2092, 2093, 2094, 2095, 2096, 2097, 2098, 2099, 2100, 2101, 2102, 2103, 2104, 2105, 2106, 2107, 2108, 2109, 2110, 2111, 2112, 2113, 2114, 2115, 2116, 2117, 2118, 2119, 2120, 2121, 2122, 2123, 2124, 2125, 2126, 2127, 2128, 2129, 2130, 2131, 2132, 2133, 2134, 2135, 2136, 2137, 2138, 2139, 2140, 2141, 2142, 2143, 2144, 2145, 2146, 2147, 2148, 2149, 2150, 2151, 2152, 2153, 2154, 2155, 2156, 2157, 2158, 2159, 2160, 2161, 2162, 2163, 2164, 2165, 2166, 2167, 2168, 2169, 2170, 2171, 2172, 2173, 2174, 2175, 2176, 2177, 2178, 2179, 2180, 2181, 2182, 2183, 2184, 2185, 2186, 2187, 2188, 2189, 2190, 2191, 2192, 2193, 2194, 2195, 2196, 2197, 2198, 2199, 2200, 2201, 2202, 2203, 2204, 2205, 2206, 2207, 2208, 2209, 2210, 2211, 2212, 2213, 2214, 2215, 2216, 2217, 2218, 2219, 2220, 2221, 2222, 2223, 2224, 2225, 2226, 2227, 2228, 2229, 2230, 2231, 2232, 2233, 2234, 2235, 2236, 2237, 2238, 2239, 2240, 2241, 2242, 2243, 2244, 2245, 2246, 2247, 2248, 2249, 2250, 2251, 2252, 2253, 2254, 2255, 2256, 2257, 2258, 2259, 2260, 2261, 2262, 2263, 2264, 2265, 2266, 2267, 2268, 2269, 2270, 2271, 2272, 2273, 2274, 2275, 2276, 2277, 2278, 2279, 2280, 2281, 2282, 2283, 2284, 2285, 2286, 2287, 2288, 2289, 2290, 2291, 2292, 2293, 2294, 2295, 2296, 2297, 2298, 2299, 2300, 2301, 2302, 2303, 2304, 2305, 2306, 2307, 2308, 2309, 2310, 2311, 2312, 2313, 2314, 2315, 2316, 2317, 2318, 2319, 2320, 2321, 2322, 2323, 2324, 2325, 2326, 2327, 2328, 2329, 2330, 2331, 2332, 2333, 2334, 2335, 2336, 2337, 2338, 2339, 2340, 2341, 2342, 2343, 2344, 2345, 2346, 2347, 2348, 2349, 2350, 2351, 2352, 2353, 2354, 2355, 2356, 2357, 2358, 2359, 2360, 2361, 2362, 2363, 2364, 2365, 2366, 2367, 2368, 2369, 2370, 2371, 2372, 2373, 2374, 2375, 2376, 2377, 2378, 2379, 2380, 2381, 2382, 2383, 2384, 2385, 2386, 2387, 2388, 2389, 2390, 2391, 2392, 2393, 2394, 2395, 2396, 2397, 2398, 2399, 2400, 2401, 2402, 2403, 2404, 2405, 2406, 2407, 2408, 2409, 2410, 2411, 2412, 2413, 2414, 2415, 2416, 2417, 2418, 2419, 2420, 2421, 2422, 2423, 2424, 2425, 2426, 2427, 2428, 2429, 2430, 2431, 2432, 2433, 2434, 2435, 2436, 2437, 2438, 2439, 2440, 2441, 2442, 2443, 2444, 2445, 2446, 2447, 2448, 2449, 2450, 2451, 2452, 2453, 2454, 2455, 2456, 2457, 2458, 2459, 2460, 2461, 2462, 2463, 2464, 2465, 2466, 2467, 2468, 2469, 2470, 2471, 2472, 2473, 2474, 2475, 2476, 2477, 2478, 2479, 2480, 2481, 2482, 2483, 2484, 2485, 2486, 2487, 2488, 2489, 2490, 2491, 2492, 2493, 2494, 2495, 2496, 2497, 2498, 2499, 2500, 2501, 2502, 2503, 2504, 2505, 2506, 2507, 2508, 2509, 2510, 2511, 2512, 2513, 2514, 2515, 2516, 2517, 2518, 2519, 2520, 2521, 2522, 2523, 2524, 2525, 2526, 2527, 2528, 2529, 2530, 2531, 2532, 2533, 2534, 2535, 2536, 2537, 2538, 2539, 2540, 2541, 2542, 2543, 2544, 2545, 2546, 2547, 2548, 2549, 2550, 2551, 2552, 2553, 2554, 2555, 2556, 2557, 2558, 2559, 2560, 2561, 2562, 2563, 2564, 2565, 2566, 2567, 2568, 2569, 2570, 2571, 2572, 2573, 2574, 2575, 2576, 2577, 2578, 2579, 2580, 2581, 2582, 2583, 2584, 2585, 2586, 2587, 2588, 2589, 2590, 2591, 2592, 2593, 2594, 2595, 2596, 2597, 2598, 2599, 2600, 2601, 2602, 2603, 2604, 2605, 2606, 2607, 2608, 2609, 2610, 2611, 2612, 2613, 2614, 2615, 2616, 2617, 2618, 2619, 2620, 2621, 2622, 2623, 2624, 2625, 2626, 2627, 2628, 2629, 2630, 2631, 2632, 2633, 2634, 2635, 2636, 2637, 2638, 2639, 2640, 2641, 2642, 2643, 2644, 2645, 2646, 2647, 2648, 2649, 2650, 2651, 2652, 2653, 2654, 2655, 2656, 2657, 2658, 2659, 2660, 2661, 2662, 2663, 2664, 2665, 2666, 2667, 2668, 2669, 2670, 2671, 2672, 2673, 2674, 2675, 2676, 2677, 2678, 2679, 26

- Exemples:

-128	64	32	16	8	4	2	1
1	0	0	0	0	0	1	1

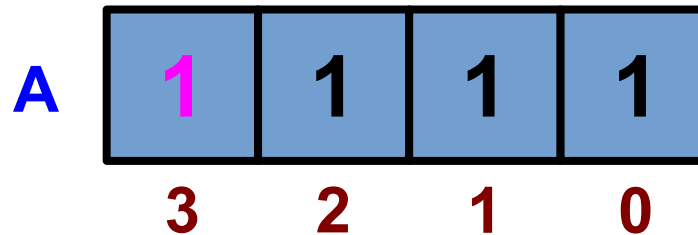
$$-128 \qquad \qquad \qquad +2 \quad +1 \quad = \underline{-125}$$

-128	64	32	16	8	4	2	1
1	0	0	0	1	0	0	0

$$\underline{-120} = -128 \quad +8$$

Representação de Números Inteiros

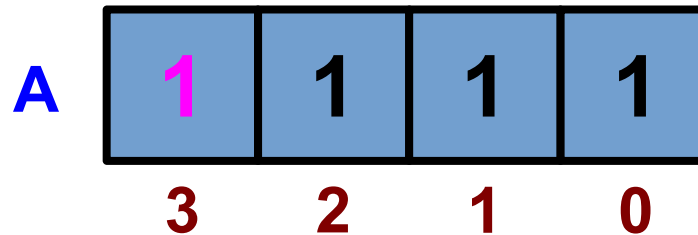
- Representação em complemento de dois



- Para 4 bits, quantos e quais valores **negativos** são possíveis representar usando complemento de dois?

Representação de Números Inteiros

- Representação em complemento de dois



- Para 4 bits, quantos e quais valores **negativos** são possíveis representar usando complemento de dois?

- 2^{n-1} valores = 2^3 valores = 8 valores

- $-1_{10} \rightarrow -2^{n-1}_{10}$ $-1_{10} \rightarrow -2^3_{10}$
 $-1_{10} \rightarrow -8_{10}$

Zero é positivo!

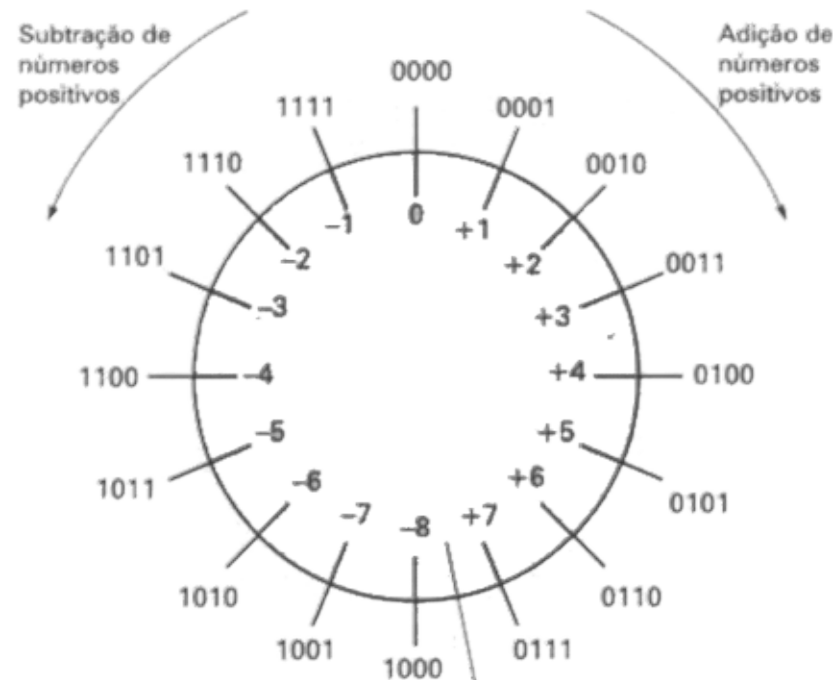
Representação de Números Inteiros

- Representação em complemento de dois

4 bits

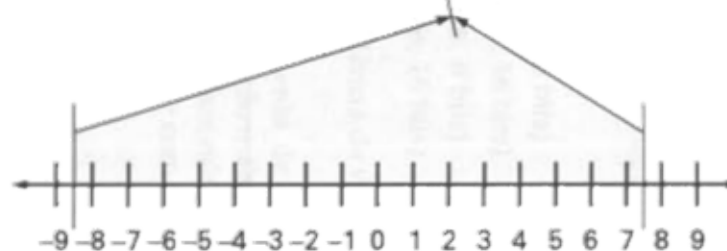
- $2^3 \rightarrow + (2^3 - 1)$

- 8 $\rightarrow + 7$



O zero é positivo!

Por isso o lado positivo representa um valor a menos

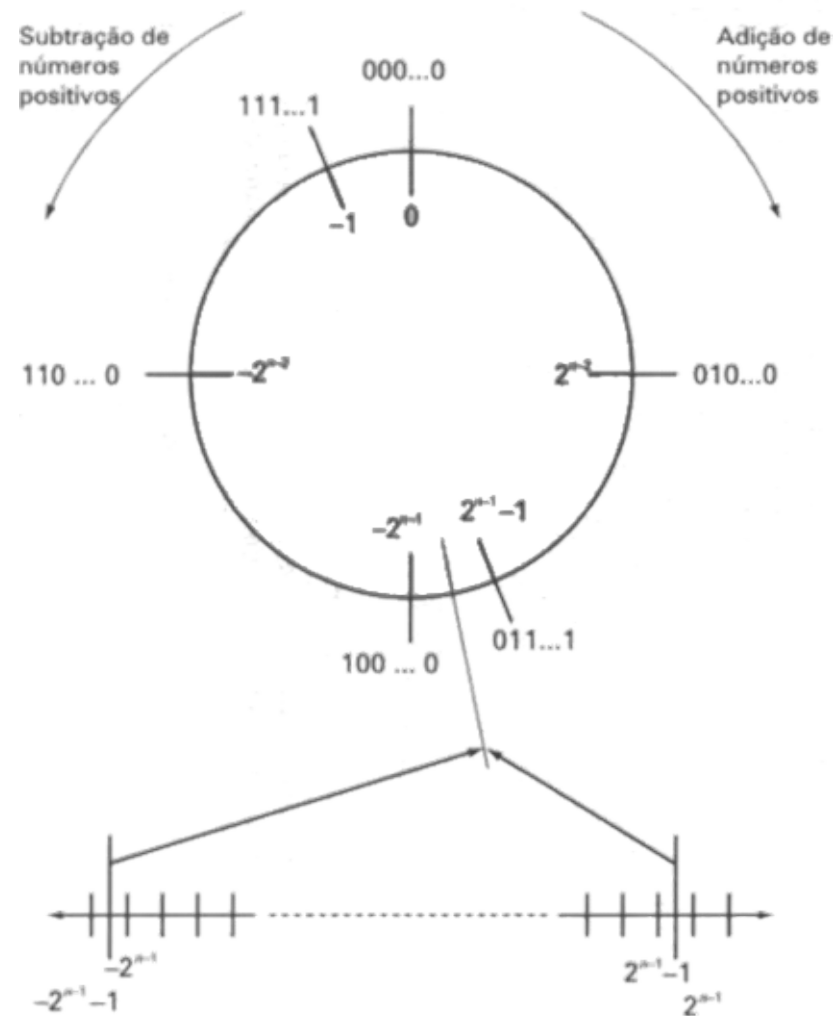


Representação de Números Inteiros

- Representação em complemento de dois

n bits

$$-2^{n-1} \rightarrow + (2^{n-1} - 1)$$

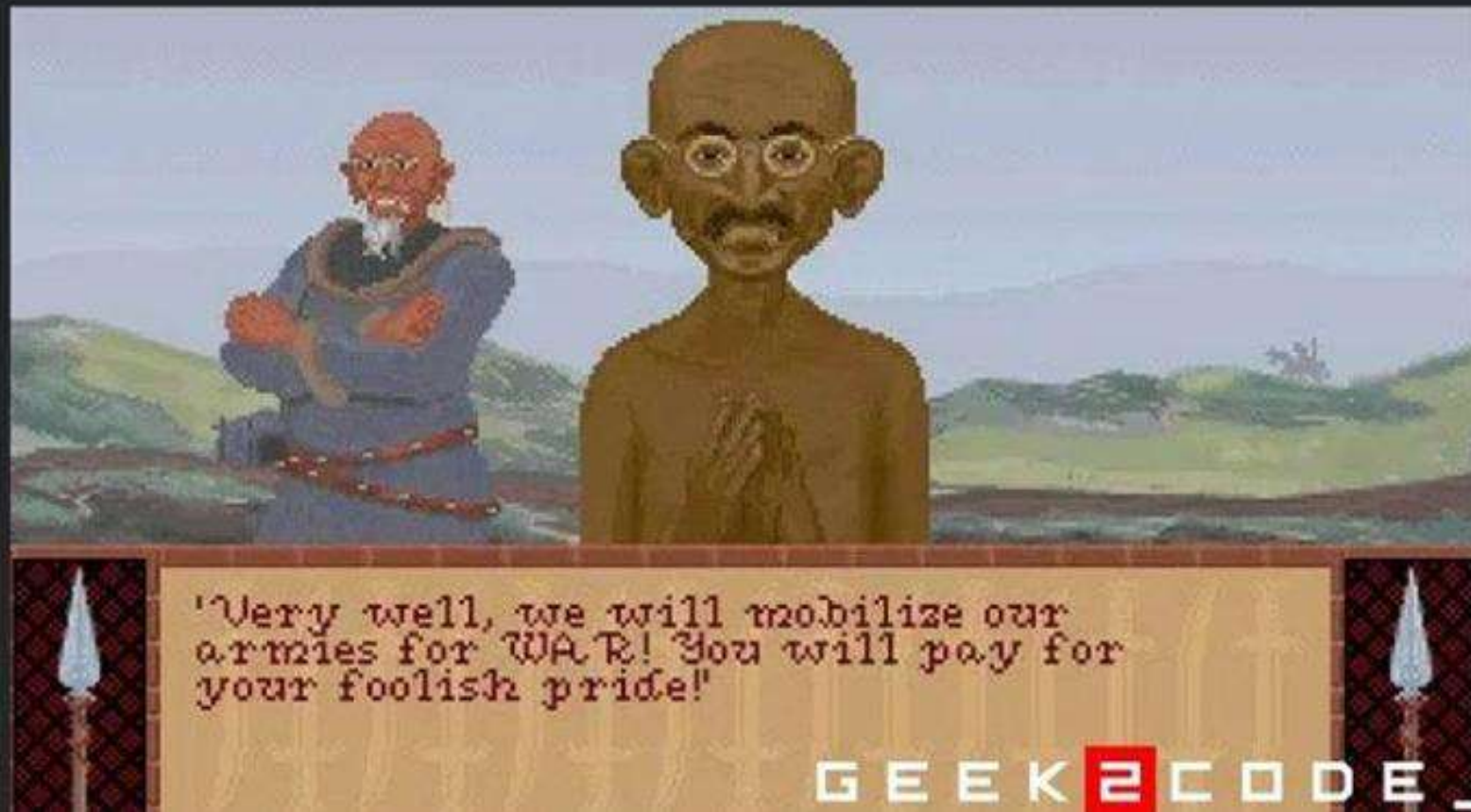


Você conhece o segredo do
Bug
do Gandhi no
Civilization I?

GEEK
CODE -

Civilization I foi um jogo de estratégia em turnos lançado em 1991, onde seu objetivo é o de conquistar um império concorrendo com outras civilizações.

Na sua primeira versão, **Gandhi** foi projetado para ser o personagem mais pacífico do jogo, assim, **era o único a ter agressividade com valor 1.**



Acontece que uma das possíveis ações do jogo era aceitar a **Democracia**, e **isso diminuía em 2 pontos a agressividade de todos os personagens.**

O erro foi que os **programadores não anteciparam que a agressividade poderia ser um valor negativo.**

Para armazenar a agressividade eles usaram uma variável de **1 byte** que só esperava valores **inteiros entre 0 a 255.**



Mas Gandhi, após aceitar a democracia, deveria ter agressividade **-1...**

O que acontece quando você passa um valor negativo a uma variável que só espera valores positivos?

Quando queremos armazenar um número inteiro, funciona assim:



Assim, com 1 byte conseguimos representar de -128 a +127.

Mas se você não espera números negativos, por que gastar 1 bit com sinal? Você pode usar todos para armazenar o valor, e agora você consegue representar de 0 a 255.

Você reparou que conseguir representar -128 números negativos e somente 127 positivos? Isso é porque usamos algo chamado "**Complemento de 2**" para representar números negativos. Isso é pra evitar que você tenha 0 positivo e 0 negativo e "gaste" uma representação sem utilidade.

Resumindo, -1 em Complemento de 2 é representado em 1 byte por **11111111**.

Mas, se você estiver considerando todos os valores como significativos, como você lê esse número? Você interpreta esse número como se fosse **255**!

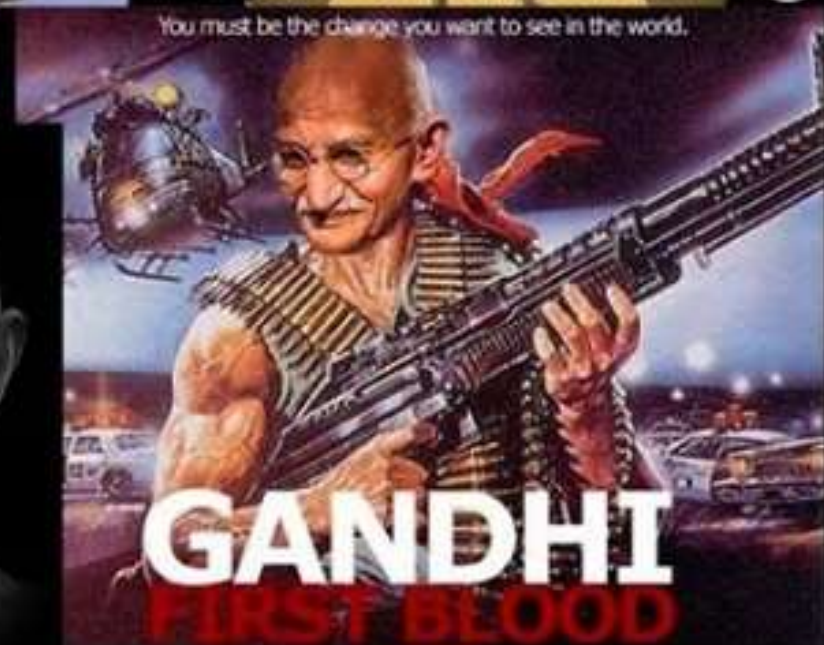
E assim Gandhi fica super-ultra agressivo!

O Bug do comportamento agressivo virou um easter-egg e foi mantido nas versões seguintes, dessa vez intencionalmente, dando origem a vários memes na comunidade.



"The world is the problem; the atomic bomb is the answer."

- Mahatma Gandhi



GANDHI
FIRST BLOOD
GEEK2CODE_

Representação de Números Inteiros

- Representação em complemento de dois

Faixa de valores representáveis	-2^{n-1} a $2^{n-1} - 1$
Número de representações para zero	1
Negação	Pegue o complemento booleano de cada bit do número positivo correspondente e então some 1 ao padrão de bits resultante, tratado como um número inteiro sem sinal.
Expansão do número de bits	Acrescente posições de bit à esquerda e preencha esses bits com o valor do bit de sinal original.
Regra de <i>overflow</i>	Se dois números com mesmo sinal (ambos positivos ou ambos negativos) forem somados, ocorrerá <i>overflow</i> apenas se o resultado tiver sinal oposto.
Regra de subtração	Para subtrair B de A , pegue o complemento de dois de B e some-o com A .

Representação de Números Inteiros

- Representação em complemento de dois
 - Negação
 - Como seria a negação do valor 5_{10} em complemento de dois?

Representação de Números Inteiros

- Representação em complemento de dois

- Negação

- Como seria a negação do valor 5_{10} em complemento de dois?

- $5_{10} = 0 \quad 1 \quad 0 \quad 1$

- Complemento de $5_{10} = 1 \quad 0 \quad 1 \quad 0$

- Complemento de $5_{10} + 1 = 1 \quad 0 \quad 1 \quad 1$

- $-5_{10} = 1 \quad 0 \quad 1 \quad 1$

Representação de Números Inteiros

- Representação em complemento de dois

- Negação

- E para o valor 0_{10} com 4 bits?

- $0_{10} = 0 \quad 0 \quad 0 \quad 0$

- Complemento de $0_{10} = 1 \quad 1 \quad 1 \quad 1$

- Complemento de $0_{10} + 1 =$ **1** $0 \quad 0 \quad 0 \quad 0$

- Única representação para zero

← Ignorado

Representação de Números Inteiros

- Representação em complemento de dois

Faixa de valores representáveis	-2^{n-1} a $2^{n-1} - 1$
Número de representações para zero	1
Negação 	Pegue o complemento booleano de cada bit do número positivo correspondente e então some 1 ao padrão de bits resultante, tratado como um número inteiro sem sinal.
Expansão do número de bits	Acrescente posições de bit à esquerda e preencha esses bits com o valor do bit de sinal original.
Regra de <i>overflow</i>	Se dois números com mesmo sinal (ambos positivos ou ambos negativos) forem somados, ocorrerá <i>overflow</i> apenas se o resultado tiver sinal oposto.
Regra de subtração	Para subtrair B de A , pegue o complemento de dois de B e some-o com A .

Representação de Números Inteiros

- Representação em complemento de dois
 - Expansão do número de bits
 - Acrescentar bits a esquerda com o mesmo valor do bit mais a esquerda
 - $11_2 = 111_2 = 1111_2$
 - $11_2 = -2 + (1 * 2^0) = -2+1 = -1$
 - $111_2 = -4 + (1 * 2^1) + (1 * 2^0) = -4+2+1 = -1$
 - $1111_2 = -8 + (1 * 2^2) + (1 * 2^1) + (1 * 2^0) = -1$

Representação de Números Inteiros

- Representação em complemento de dois

Faixa de valores representáveis		-2^{n-1} a $2^{n-1} - 1$
Número de representações para zero		1
Negação		Pegue o complemento booleano de cada bit do número positivo correspondente e então some 1 ao padrão de bits resultante, tratado como um número inteiro sem sinal.
Expansão do número de bits		Acrescente posições de bit à esquerda e preencha esses bits com o valor do bit de sinal original.
Regra de <i>overflow</i>		Se dois números com mesmo sinal (ambos positivos ou ambos negativos) forem somados, ocorrerá <i>overflow</i> apenas se o resultado tiver sinal oposto.
Regra de subtração		Para subtrair B de A , pegue o complemento de dois de B e some-o com A .

Representação de Números Inteiros

- Representação em complemento de dois

- Overflow

- Qual o valor de $0111_2 + 0111_2$?

$$\begin{array}{rcccccl} & 0 & 1 & 1 & 1 & \rightarrow & +7 \\ + & 0 & 1 & 1 & 1 & \rightarrow & +7 \\ \hline & \textcolor{red}{1} & 1 & 1 & 0 & \rightarrow & -8 \end{array}$$

- $1110_2 = -8_{10}$
 - $01110_2 = +14_{10}$

Representação de Números Inteiros

- Representação em complemento de dois

Faixa de valores representáveis		-2^{n-1} a $2^{n-1} - 1$
Número de representações para zero		1
Negação		Pegue o complemento booleano de cada bit do número positivo correspondente e então some 1 ao padrão de bits resultante, tratado como um número inteiro sem sinal.
Expansão do número de bits		Acrescente posições de bit à esquerda e preencha esses bits com o valor do bit de sinal original.
Regra de <i>overflow</i>		Se dois números com mesmo sinal (ambos positivos ou ambos negativos) forem somados, ocorrerá <i>overflow</i> apenas se o resultado tiver sinal oposto.
Regra de subtração		Para subtrair B de A , pegue o complemento de dois de B e some-o com A .

Representação de Números Inteiros

- Representação em complemento de dois

- Subtração

- Qual o valor de $0111_2 - 0101_2$?

- Complemento de dois de $0101_2 = 1011_2$

	0	1	1	1	→	+7
	+	1	0	1	1	→ -5
	1	0	0	1	0	→ +2

Representação de Números Inteiros

- Representação em complemento de dois

- Subtração

- Qual o valor de $1101_2 - 0101_2$?

- Complemento de dois de $0101_2 = 1011_2$

	1	1	0	1	→	-3
	1	0	1	1	→	-5
+	1	0	1	1		
	1	1	0	0	→	-8

Representação de Números Inteiros

- Representação em complemento de dois
 - Subtração
 - Operação de subtração é uma operação de adição
 - Tanto a adição como a subtração podem ser feitas no mesmo hardware de soma

Representação de Números Inteiros

- Representação em complemento de dois

Faixa de valores representáveis		-2^{n-1} a $2^{n-1} - 1$
Número de representações para zero		1
Negação		Pegue o complemento booleano de cada bit do número positivo correspondente e então some 1 ao padrão de bits resultante, tratado como um número inteiro sem sinal.
Expansão do número de bits		Acrescente posições de bit à esquerda e preencha esses bits com o valor do bit de sinal original.
Regra de <i>overflow</i>		Se dois números com mesmo sinal (ambos positivos ou ambos negativos) forem somados, ocorrerá <i>overflow</i> apenas se o resultado tiver sinal oposto.
Regra de subtração		Para subtrair B de A , pegue o complemento de dois de B e some-o com A .

Aritmética de Números Inteiros

- **Multiplicação**

- Como você implementaria uma multiplicação usando apenas adições?



Aritmética de Números Inteiros

- **Multiplicação**

- Como você implementaria uma multiplicação usando apenas adições?

- Exemplo: $5 * 3$

```
int resultado = 0;
for( int x=1; x<=3; x++ )
{
    resultado += 5;
}
```

Aritmética de Números Inteiros

- **Multiplicação**

- É uma operação complexa, seja implementada em hardware ou software
- A multiplicação de **n bits** resulta em um produto com até **2n bits** de comprimento.

- Em decimal: $9 * 9 = 18$
9.801

$$99 * 99 =$$

Aritmética de Números Inteiros

- **Multiplicação**

- O produto total é obtido somando-se os produtos parciais

$$\begin{array}{r} 12 \\ \times 14 \\ \hline 48 \quad \longleftarrow 12 \times 4 \\ + 12 \quad \longleftarrow 12 \times 1 \\ \hline 168 \end{array}$$

Aritmética de Números Inteiros

- **Multiplicação Binária**

- Quando o bit do multiplicador é 0:
 - Produto parcial é 0
- Quando o bit do multiplicador é 1:
 - Produto parcial é o próprio multiplic



1011	Multiplicando (11)
× 1101	Multiplicador (13)
1011	} Produtos parciais
0000	
1011	
1011	
10001111	Produto (143)

8 caracteres



Aritmética de Números Inteiros

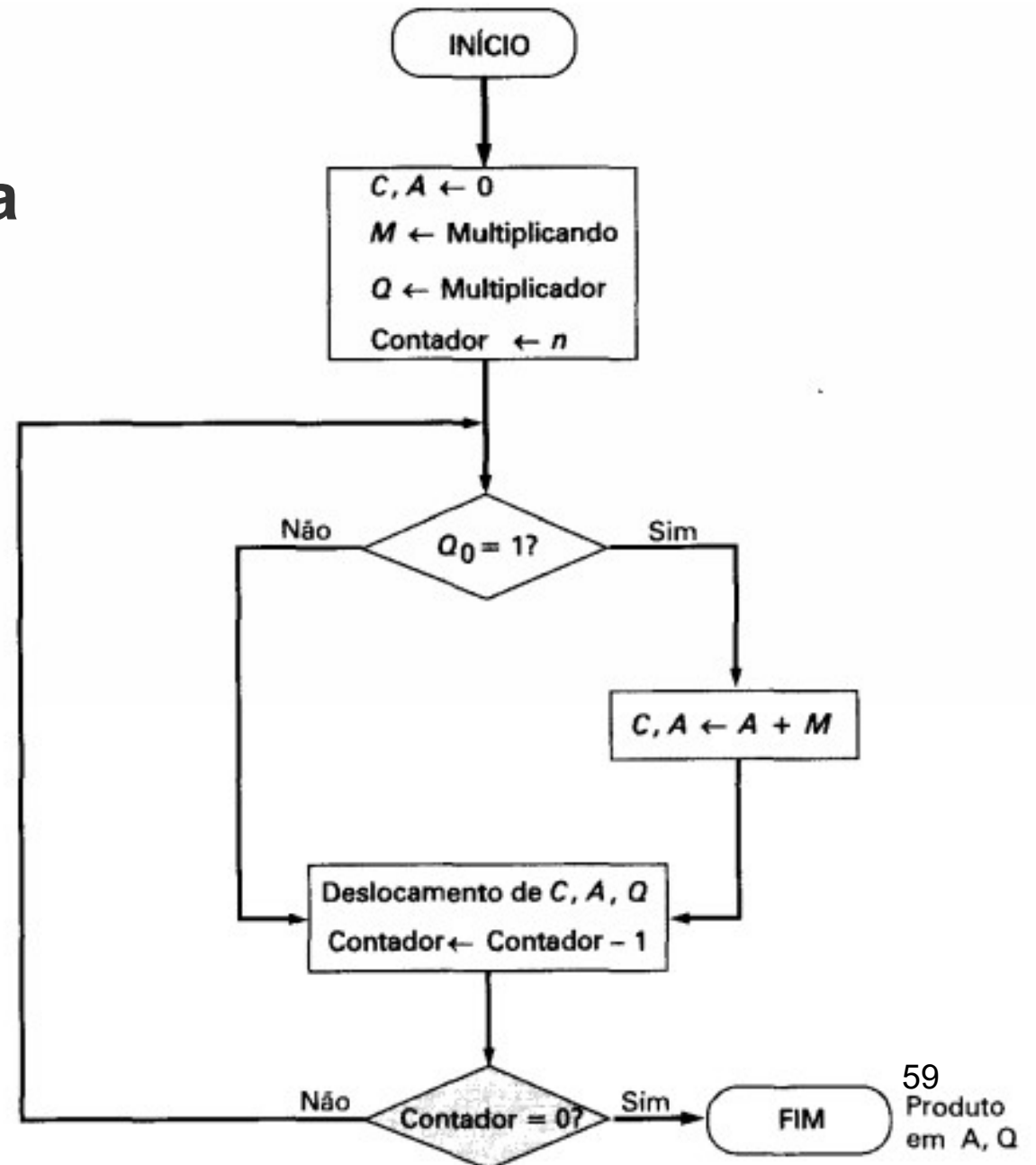
- **Multiplicação Binária**

- Para cada 1 no multiplicador:
 - **Uma** operação de soma e **um** deslocamento
- Para cada 0 no multiplicador:
 - Apenas **um** deslocamento é necessário

1011	Multiplicando (11)
× 1101	Multiplicador (13)
1011	} Produtos parciais
0000	
1011	
1011	
10001111	Produto (143)

Aritmética de Números Inteiros

- Multiplicação Binária



Aritmética de Números Inteiros

- Multiplicação Binária**

1011	Multiplicando (11)
× 1101	Multiplicador (13)
1011	} Produtos parciais
0000	
1011	
1011	
10001111	Produto (143)

C	A	Q	M		
0	0000	1101	1011	Valores iniciais	
0	1011	1101	1011	Adição	} Primeiro Ciclo
0	0101	1110	1011	Deslocamento	
0	0010	1111	1011	Deslocamento	} Segundo Ciclo
0	1101	1111	1011	Adição	
0	0110	1111	1011	Deslocamento	} Terceiro Ciclo
1	0001	1111	1011	Adição	
0	1000	1111	1011	Deslocamento	} Quarto Ciclo

Aritmética de Números Inteiros

- **Multiplicação Binária em Complemento de Dois**
 - Muda a forma da multiplicação

Sem considerar os sinais

$$\begin{array}{r} 1001 \quad (9) \\ \times 0011 \quad (3) \\ \hline 00001001 \\ 00010010 \\ \hline 00011011 \end{array} \quad \begin{array}{l} 1001 \times 2^0 \\ 1001 \times 2^1 \\ (27) \end{array}$$

Complemento de Dois

$$\begin{array}{r} 1001 \quad (-7) \\ \times 0011 \quad (3) \\ \hline 11111001 \\ 11110010 \\ \hline 11101011 \end{array} \quad \begin{array}{l} (-7) \times 2^0 = (-7) \\ (-7) \times 2^1 = (-14) \\ (-21) \end{array}$$

Usando 8 bits para representar os valores
Usamos a expansão do número de bits
Deve-se alinhar os bits de sinal

Aritmética de Números Inteiros

- **Multiplicação Binária em Complemento de Dois**
 - Utiliza-se o algoritmo de Booth para implementar a multiplicação
 - Bastante complexa 
 - Mais rápida que a maneira convencional 



Aritmética de Números Inteiros

- **Divisão Binária**

- Mais complexa que a multiplicação
- Implementada como repetidas execuções de adição, subtração e deslocamento

100

Representação de Números de Ponto Flutuante

- Usando notação de ponto fixo:
 - Ex Complemento de Dois: 1 Byte (8 bits)
 - 1 bit para o sinal
 - 4 bits para valores inteiros
 - 3 bits para valores fracionários



Representação de Números de Ponto Flutuante

- Usando notação de ponto fixo:
 - Impossível representar valores muito grandes
 - Impossível representar frações muito pequenas
 - Na divisão ou multiplicação pode haver grandes perdas





Representação de Números de Ponto Flutuante

- Uso da notação científica
 - $976.000.000.000.000 \rightarrow 9,76 \times 10^{14}$
 - $0,0000000000000000976 \rightarrow 9,76 \times 10^{-14}$
 - A vírgula é deslizada para uma posição conveniente
 - Formulação genérica:



Representação de Números de Ponto Flutuante

- Uso da notação científica
 - Precisamos armazenar apenas 3 valores:
 - Sinal
 - Mantissa
 - Expoente Polarizado
 - A base é sempre constante no computador, logo não precisamos armazenar

Representação de Números de Ponto Flutuante

- Normalização
 - A Mantissa precisa estar entre:
 - $1 \leq \text{Mantissa} < \text{Base}$
 - Exemplo
 - $976.000.000.000.000 \rightarrow 9,76 \times 10^{14}$
 - $1,011011 \times 2^{110}_2$
- Padrão IEEE 754

Representação de Números de Ponto Flutuante

- Uso da notação científica
 - Precisamos armazenar apenas 3 valores:
 - Sinal
 - Mantissa
 - Expoente Polarizado
 - Exemplo: Palavra (32 bits)





Representação de Números de Ponto Flutuante

- Uso da notação científica
 - Sinal
 - Apenas 1 bit
 - 0 para valores positivos
 - 1 para valores negativos



Representação de Números de Ponto Flutuante

- Uso da notação científica
 - Mantissa
 - Existem várias formas de representar um mesmo número
 - $11,10 \times 2^5 = 111 \times 2^4 = 1,110 \times 2^6$
 - Padronização da representação:



Representação de Números de Ponto Flutuante

- Uso da notação científica
 - Mantissa
 - Padronização da representação:
 - Todos os números diferente de zero começam com 1,bbbb
 - **Não se armazena o primeiro 1 da mantissa**

Representação de Números de Ponto Flutuante

- Uso da notação científica
 - Mantissa
 - Ganhamos 1 dígito na mantissa
 - Com 23 bits conseguimos representar 24 bits



Representação de Números de Ponto Flutuante

- Uso da notação científica
 - Mantissa
 - $0,1_2 = 0 + 2^{-1} = 0,5_{10}$
 - $2^{-1} = \frac{1}{2} = 0,5$

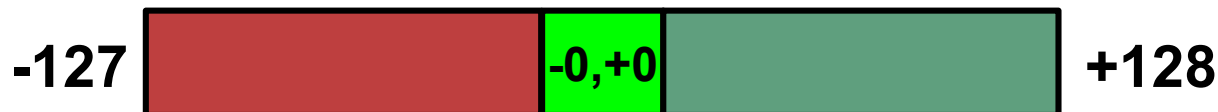


Representação de Números de Ponto Flutuante

- Uso da notação científica
 - Mantissa
 - $0,111\dots 1_2$ tende a 1_{10}
 - Menor valor para a **mantissa** será 1,0

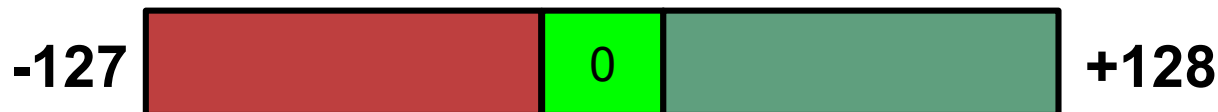
Representação de Números de Ponto Flutuante

- **Representação Polarizada**
 - Não existe dígito de sinal
 - Metade dos valores serão positivos e a outra metade negativa
 - Polo positivo, Polo negativo
 - Exemplo: Com 8 bits → 256 valores
 - 2^7 valores positivos
 - 2^7 valores negativos



Representação de Números de Ponto Flutuante

- Representação Polarizada
 - Valores contínuos
 - Exemplo: Com 8 bits → 256 valores
 - 0000 0000 = -127 (Menor valor)
 - 0111 1111 = 0
 - 1111 1111 = +128 (Maior valor)





Representação de Números de Ponto Flutuante

- **Representação Polarizada**
 - Grande vantagem:
 - Comparação entre número mais fácil
 - $0100 > 0001$
 - $1010 < 1100$

Representação de Números de Ponto Flutuante

- Representação Polarizada
 - Polarização:
 - Valor constante $C_p = (2^{(N-1)} - 1)$
 - N é a quantidade de bits usada
 - $\text{Valor}_{\text{polarizado}} = C_p + \text{Valor}_{\text{original}}$

Pode-se interpretar essa fórmula como se todos os valores partissem da metade
Para valores negativos a tendencia é zerar todos os bits
Para valores positivos a tendencia é que todos os bits virem 1

Representação de Números de Ponto Flutuante

- Converta para um número polarizado em binário com 4 bits:

- 6_{10}

- 2_{10}

- -3_{10}

- $C_p = (2^{(N-1)} - 1)$
- $\text{Valor}_p = C_p + \text{Valor}_o$

Representação de Números de Ponto Flutuante

- Converta para um número polarizado em binário com 4 bits:

- $6_{10} \rightarrow 0110_2$

- $\text{Valor}_p = 0111_2 + 0110_2 = 1101_2$

- $2_{10} \rightarrow 0010_2$

- $\text{Valor}_p = 0111_2 + 0010_2 = 1001_2$

- $-3_{10} \rightarrow -0011_2$

- $\text{Valor}_p = 0111_2 - 0011_2 = 0100_2$

- $C_p = (2^{(N-1)} - 1) \rightarrow$

- $C_p = (2^{(4-1)} - 1) \rightarrow$
 $\text{Valor}_p = C_p + \text{Valor}_o$

- $C_p = 7 \rightarrow 0111_2$

Representação de Números de Ponto Flutuante

- Uso da notação científica
 - Expoente Polarizado
 - Adicionar ao expoente original o valor $(2^7 - 1)$ para se obter o valor polarizado
 - $N = 8$ bits
 - $(2^{N-1}-1) = 2^7 - 1 = 127_{10} = 0111\ 1111_2$



Representação de Números de Ponto Flutuante

- Exemplos:



– 1,01010001 x 2¹⁰¹⁰⁰

0 10010011 01010001000000000000000000000000

→ Primeiro bits não é representado

0111 1111	←	2 ⁷ - 1
+ 0001 0100	←	Expoente de entrada
1001 0011		

Representação de Números de Ponto Flutuante

- Exemplos:



- 1,11010001 x 2¹⁰¹⁰⁰

1 **10010011** **11010001**100000000000000000000000

└─ Primeiro bits não é representado

0111 1111	←	2 ⁷ - 1
+ 0001 0100	←	Expoente de entrada
1001 0011		

Representação de Números de Ponto Flutuante

- Exemplos:

– $1,11010001 \times 2^{-10100}$

0 01101011 110100010000000000000000

→ Primeiro bits não é representado

$$\begin{array}{r}
 0111 \ 1111 \leftarrow 2^7 - 1 \\
 - \underline{0001 \ 0100} \leftarrow \text{Expoente de entrada} \\
 0110 \ 1011
 \end{array}$$

1 BIT

8 BITS

23 BITS



Representação de Números de Ponto Flutuante

- Exemplos:



- 1,11010001 x 2⁻¹⁰¹⁰⁰

1 **01101011** **101000100000000000000000**

└─ Primeiro bits não é representado

0111 1111	←	2 ⁷ - 1
- 0001 0100	←	Expoente de entrada
0110 1011		

Representação de Números de Ponto Flutuante

- Representatividade para **Inteiros**
 - Complemento de Dois
 - 1 bit de sinal e 31 de magnitude
 - 2^{31} valores positivos e 2^{31} valores negativos
 - -2^{31} a -1
 - 0 a $(2^{31} - 1)$

Representação de Números de Ponto Flutuante

- Representatividade para valores **Reais**
 - Ponto Flutuante
 - Expoentes variando de -127 a +127
 - Mantissa com 24 bits
 - $-(2 - 2^{-24})$ → Tende a - 1,0
 - -1,0 → Mantissa completamente zerada
 - +1,0 → Mantissa completamente zerada
 - $+(2 - 2^{-24})$ → Tende a + 1,0



Representação de Números de Ponto Flutuante

- Representatividade para valores **Reais**
 - Observou que não existe o valor zero absoluto nos número reais?
 - **Nunca** faça:
 - `if ( limite == 0.0 )`
 - **Faça:**
 - `if ( limite >= (0.0 – getTolerancia()) )`





Aritmética de Números de Ponto Flutuante

- Problemas podem ocorrer:
 - **Overflow no expoente:** expoente positivo excede o maior valor possível para um expoente
 - Número é muito grande, pode ser representado como **$+\infty$ OU $-\infty$**
 - **Underflow no expoente:** expoente negativo é menor que o menor valor possível para um expoente.
 - Número é muito pequeno, pode ser representado como **0**



Aritmética de Números de Ponto Flutuante

- Problemas podem ocorrer:
 - **Underflow na mantissa:** pode ocorrer o transbordo de dígitos para a extremidade direita, pode tentar corrigir com um novo alinhamento do número
 - **Overflow na mantissa:** pode ocorrer o transbordo de dígitos para a extremidade esquerda (“vai-um”), pode tentar corrigir com um novo alinhamento do número
 - **NaN (Not a Number):** resultado de operação inválida (ex: $0/0$, $\infty - \infty$, raiz de negativo). O IEEE 754 representa com padrão especial de bits



Aritmética de Números de Ponto Flutuante

- ⌚ Adição de Números de Ponto Flutuante
 - **Passo 1 – Alinhamento dos expoentes:** desloca a mantissa do número com menor expoente até igualar os expoentes
 - **Passo 2 – Soma das mantissas:** soma (ou subtrai) as mantissas já alinhadas, preservando o sinal
 - **Passo 3 – Normalização do resultado:** ajusta a mantissa para o formato 1,bbbb e corrige o expoente; verifica overflow/underflow
 - Exemplo: $1,010 \times 2^2 + 1,110 \times 2^3 \rightarrow$ (alinhar) $0,101 \times 2^3$
 $+ 1,110 \times 2^3 = 10,011 \times 2^3 \rightarrow$ (norm.) $1,0011 \times 2^4$ ₉₃

Arquitetura de Computadores



Universidade
Christus

Unidade 05: Aritmética de Computador

Daniel Teixeira

prof.danielnt@gmail.com
dant25.github.io/professor